

1 Tier architecture

- ui and backend are written in same file.

2 tier architecture

- It have 2 part first one is frontend and second one is backend(It have database also)

3 tier architecture

- it has
 - ui part
 - server
 - database.

API (Application programming interface)

- Api is nothing just medium between two medium

REST (Representational State transfer) - Rule how to communicate between two midumes.

fetch() will get this two object

1. header
2. body

- in the server have response object
- response have

1. header (Having metadata)

- Its an object
- "content-type"
- Type of data (json, utf-8, video, audio etc...)
- length of the content
- when last modified.
- cache-control ("max-age - 60" - it means it will accessible for 1 hour only)

2. Body (it contains actual data)

- its an object
- It contains actual data.
- data may be Array or Object
- If we fetch Multiple users data then it will come in the form of Array.
- If we fetch single user data then it came in the form of Object.

- If we can't convert the data in json format then it is not readable to human.
- If we want promise as return then don't make async function but if we want to proper data then make function async.
- async function always returns promise by-default.

Fetch API - Complete Guide

Table of Contents

1. [Introduction](#)
 2. [Architecture Types](#)
 3. [API & REST Basics](#)
 4. [Fetch API Response Structure](#)
 5. [Fetch API Syntax & Methods](#)
 6. [Practical Examples](#)
 7. [HTTP Methods with Fetch](#)
 8. [Error Handling](#)
 9. [Interview Questions](#)
 10. [Practical Coding Questions](#)
-

Introduction

The **Fetch API** is a modern JavaScript interface for making HTTP requests. It provides a cleaner and more powerful way to interact with servers compared to the older XMLHttpRequest.

Key Features:

- Promise-based API
 - Supports async/await syntax
 - More readable and maintainable code
 - Built-in JSON parsing
 - Better error handling
-

Architecture Types

1-Tier Architecture

- **Definition:** UI and backend are written in the same file
- **Example:** Simple HTML file with embedded JavaScript and no external server
- **Use Case:** Very small applications, prototypes

```
// Everything in one file
<!DOCTYPE html>
<html>
<body>
  <button onclick="alert('Hello')">Click</button>
  <script>
    // UI + Logic in same file
  </script>
</body>
</html>
```

2-Tier Architecture

- **Definition:** Has 2 parts
 - Frontend (UI)
 - Backend (Server + Database combined)
- **Example:** React frontend talking directly to a Node.js server with embedded database queries



3-Tier Architecture

- **Definition:** Has 3 separate layers
 - UI Part (Frontend)
 - Server (Backend/API)
 - Database (Separate layer)
- **Example:** Modern web applications (React + Express API + MongoDB)



API & REST Basics

What is an API?

API (Application Programming Interface) is a medium/interface between two systems that allows them to communicate.

Simple Analogy:

- API is like a waiter in a restaurant
- You (client) tell the waiter what you want
- Waiter takes your order to the kitchen (server)
- Kitchen prepares food (processes request)
- Waiter brings food back to you (response)

What is REST?

REST (Representational State Transfer) - A set of rules/principles for how client and server should communicate.

REST Principles:

1. Client-Server separation
2. Stateless (each request is independent)
3. Cacheable responses
4. Uniform interface (standard HTTP methods)
5. Layered system

HTTP Methods in REST:

- **GET** - Retrieve data
 - **POST** - Create new data
 - **PUT** - Update existing data
 - **PATCH** - Partially update data
 - **DELETE** - Delete data
-

Fetch API Response Structure

When you make a `fetch()` request, you receive a **Response Object** that contains two main parts:

1. Headers (Metadata)

Headers contain metadata about the response.

```
const response = await fetch(url);

// Accessing headers
console.log(response.headers.get('content-type'));
console.log(response.headers.get('content-length'));
```

Common Headers:

- **content-type**: Type of data (json, text/html, video, audio, etc.)
- **content-length**: Size of the response body

- `last-modified`: When the resource was last modified
- `cache-control`: Caching directives (e.g., "max-age=3600" means cache for 1 hour)
- `authorization`: Authentication credentials

Header Object Properties:

```
{
  "content-type": "application/json; charset=utf-8",
  "content-length": "1024",
  "cache-control": "max-age=60",
  "last-modified": "Mon, 13 Jan 2025 10:30:00 GMT"
}
```

2. Body (Actual Data)

The body contains the actual data returned from the server.

```
const response = await fetch(url);
const data = await response.json(); // Parse JSON body
```

Body Types:

- **Array**: When fetching multiple records

```
[
  { id: 1, name: "User1" },
  { id: 2, name: "User2" }
]
```

- **Object**: When fetching a single record

```
{
  id: 1,
  name: "User1",
  email: "user1@example.com"
}
```

Important:

- Data must be converted to JSON format to be human-readable
 - Use `.json()` method to parse the response body
 - `.json()` returns a Promise
-

Fetch API Syntax & Methods

Basic Syntax

```
fetch(url, options)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

Response Methods

```
const response = await fetch(url);

// Parse response body in different formats
const jsonData = await response.json();    // Parse as JSON
const textData = await response.text();     // Parse as text
const blobData = await response.blob();     // Parse as Blob (files)
const formData = await response.formData(); // Parse as FormData
```

Response Properties

```
const response = await fetch(url);

console.log(response.ok);           // Boolean: true if status 200-299
console.log(response.status);       // HTTP status code (200, 404, 500, etc.)
console.log(response.statusText);   // Status message ("OK", "Not Found")
console.log(response.url);          // Final URL (after redirects)
console.log(response.headers);      // Headers object
```

Async vs Non-Async Functions

```
// Non-async function - Returns Promise
function getData() {
  return fetch(url).then(res => res.json());
}

// Usage
getData().then(data => console.log(data));

// Async function - Returns Promise but easier to use
async function getData() {
  const res = await fetch(url);
  const data = await res.json();
  return data; // Still returns Promise
}

// Usage
const data = await getData();
console.log(data);
```

Key Point:

- Async functions **always return a Promise** by default
 - Even if you return a plain value, it's wrapped in a Promise
 - Use `await` to get the actual data from the Promise
-

Practical Examples

Example 1: Fetching Single User Data

```
// HTML
<button id="btn">Get User</button>
<div class="container"></div>

// JavaScript
const button = document.querySelector("#btn");

button.addEventListener("click", async () => {
  const userData = await getUserData("avinash-25");

  const div = document.querySelector(".container");
  div.innerHTML = `
    <h1>${userData.name}</h1>
    
    <p>Followers: ${userData.followers}</p>
    <p>Public Repos: ${userData.public_repos}</p>
  `;
});

async function getUserData(username) {
  const url = `https://api.github.com/users/${username}`;

  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    return data; // Returns an Object
  } catch (error) {
    console.error("Error fetching user:", error);
  }
}
```

Example 2: Fetching Multiple Users (Array)

```
const button = document.querySelector("button");
const url = "https://api.github.com/users";

const getData = async () => {
  const res = await fetch(url);
  const data = await res.json(); // Returns Array of users
  return data;
}

button.addEventListener("click", async () => {
  const users = await getData();

  // Using map to create HTML for each user
  const userCards = users.map(element => {
    const { login, avatar_url } = element;

    return `
      <div class="card">
        <div class="profile">
          
        </div>
        <div class="info">
          <h2>${login}</h2>
        </div>
      </div>
    `;
  });
  userCards.join(""); // join("") removes commas between array elements

  const div = document.querySelector(".container");
  div.innerHTML = userCards;
});
```

Example 3: Fetch with Loading State

```
const button = document.querySelector("button");
const container = document.querySelector(".container");

button.addEventListener("click", async () => {
  // Show loading state
  container.innerHTML = "<p>Loading...</p>";

  try {
    const users = await getData();
    displayUsers(users);
  } catch (error) {
    container.innerHTML = `<p style="color: red;">Error: ${error.message}</p>`;
  }
});

async function getData() {
  const response = await fetch("https://api.github.com/users");

  if (!response.ok) {
    throw new Error(`Failed to fetch: ${response.status}`);
  }

  return await response.json();
}

function displayUsers(users) {
  const html = users.map(user => `
    <div class="card">
      
      <h3>${user.login}</h3>
      <a href="${user.html_url}" target="_blank">View Profile</a>
    </div>
  `).join("");

  container.innerHTML = html;
}
```

Example 4: Fetch with Query Parameters

```
async function searchUsers(query, perPage = 10) {
  const url = `https://api.github.com/search/users?q=${query}&per_page=${perPage}`;

  const response = await fetch(url);
  const data = await response.json();

  return data.items; // GitHub search returns items array
}

// Usage
const users = await searchUsers("john", 5);
console.log(users);
```

Example 5: Fetch with Headers

```
async function fetchWithAuth(url, token) {
  const response = await fetch(url, {
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    }
  });

  const data = await response.json();
  return data;
}
```

HTTP Methods with Fetch

GET Request (Default)

```
// Simple GET
const response = await fetch('https://api.example.com/users');
const data = await response.json();

// GET with query parameters
const response = await fetch('https://api.example.com/users?page=1&limit=10');
```

POST Request (Create Data)

```
async function createUser(userData) {
  const response = await fetch('https://api.example.com/users', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(userData) // Convert object to JSON string
  });

  const data = await response.json();
  return data;
}

// Usage
const newUser = {
  name: 'John Doe',
  email: 'john@example.com'
};

const result = await createUser(newUser);
```

PUT Request (Update Full Resource)

```
async function updateUser(userId, userData) {
  const response = await fetch(`https://api.example.com/users/${userId}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(userData)
  });

  return await response.json();
}

// Usage
const updatedUser = {
  name: 'John Updated',
  email: 'johnupdated@example.com'
};

await updateUser(123, updatedUser);
```

PATCH Request (Partial Update)

```
async function updateUserEmail(userId, newEmail) {
  const response = await fetch(`https://api.example.com/users/${userId}`, {
    method: 'PATCH',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ email: newEmail })
  });

  return await response.json();
}
```

DELETE Request

```
async function deleteUser(userId) {
  const response = await fetch(`https://api.example.com/users/${userId}`, {
    method: 'DELETE'
  });

  if (response.ok) {
    return { message: 'User deleted successfully' };
  } else {
    throw new Error('Failed to delete user');
  }
}
```

Complete CRUD Example

```
const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// CREATE
async function createPost(post) {
  const response = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(post)
  });
  return await response.json();
}

// READ
async function getPosts() {
  const response = await fetch(API_URL);
  return await response.json();
}

async function getPost(id) {
  const response = await fetch(`${API_URL}/${id}`);
  return await response.json();
}

// UPDATE
async function updatePost(id, post) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(post)
  });
  return await response.json();
}

// DELETE
async function deletePost(id) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: 'DELETE'
  });
  return response.ok;
}
```

Error Handling

Basic Error Handling

```

async function fetchData(url) {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Fetch error:', error.message);
    throw error; // Re-throw for caller to handle
  }
}

```

Handling Different Error Types

```

async function robustFetch(url) {
  try {
    const response = await fetch(url);

    // Check if response is ok (status 200-299)
    if (!response.ok) {
      if (response.status === 404) {
        throw new Error('Resource not found');
      } else if (response.status === 500) {
        throw new Error('Server error');
      } else if (response.status === 401) {
        throw new Error('Unauthorized');
      } else {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
    }

    const data = await response.json();
    return data;
  } catch (error) {
    // Network errors (no internet, CORS, etc.)
    if (error instanceof TypeError) {
      console.error('Network error:', error);
      throw new Error('Network connection failed');
    }

    // Other errors
    console.error('Error:', error);
    throw error;
  }
}

```

Timeout Implementation

```

async function fetchWithTimeout(url, timeout = 5000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  try {
    const response = await fetch(url, {
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    clearTimeout(timeoutId);

    if (error.name === 'AbortError') {
      throw new Error('Request timeout');
    }
    throw error;
  }
}

// Usage
try {
  const data = await fetchWithTimeout('https://api.example.com/data', 3000);
  console.log(data);
} catch (error) {
  console.error('Failed to fetch:', error.message);
}

```

Retry Logic

```

async function fetchWithRetry(url, maxRetries = 3, delay = 1000) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      return await response.json();
    } catch (error) {
      console.log(`Attempt ${i + 1} failed. Retrying...`);

      if (i === maxRetries - 1) {
        throw error; // Last attempt failed
      }

      // Wait before retrying
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}

```

Interview Questions

1. What is the Fetch API and how does it differ from XMLHttpRequest?

Answer: The Fetch API is a modern interface for making HTTP requests in JavaScript. Key differences:

- **Promise-based:** Fetch returns Promises, making it work seamlessly with async/await
- **Cleaner syntax:** More readable and less verbose than XMLHttpRequest
- **No callback hell:** Avoids nested callbacks through Promise chaining
- **Better error handling:** Separates network errors from HTTP errors
- **Supports modern features:** Built-in JSON parsing, Request/Response objects

```
// XMLHttpRequest (Old way)
const xhr = new XMLHttpRequest();
xhr.open('GET', url);
xhr.onload = function() {
  const data = JSON.parse(xhr.responseText);
};
xhr.send();

// Fetch API (Modern way)
const response = await fetch(url);
const data = await response.json();
```

2. Explain the difference between 1-tier, 2-tier, and 3-tier architecture.

Answer:

- **1-Tier:** Everything in one file (UI + logic + data). Example: Simple HTML file with embedded JavaScript.
- **2-Tier:** Frontend and Backend combined with database. Example: React app directly querying a Node.js server with embedded database queries.
- **3-Tier:** Separate layers:
 - Presentation Layer (UI)
 - Application Layer (Server/API)
 - Data Layer (Database)

Benefits of 3-tier: Better separation of concerns, easier maintenance, scalability, and security.

3. What is REST and what are its key principles?

Answer: REST (Representational State Transfer) is an architectural style for designing networked applications.

Key Principles:

1. **Client-Server separation:** Independent development and scaling
2. **Stateless:** Each request contains all information needed
3. **Cacheable:** Responses should indicate if they can be cached
4. **Uniform Interface:** Standard HTTP methods (GET, POST, PUT, DELETE)
5. **Layered System:** Client doesn't know if connected directly to server or through intermediaries

4. What does the response object contain when using fetch()?

Answer: The Response object contains two main parts:

1. Headers (Metadata):

- `content-type`: Data format (JSON, HTML, etc.)
- `content-length`: Size of response
- `cache-control`: Caching instructions
- `last-modified`: When resource was last updated

2. Body (Actual Data):

- Contains the actual data from the server
- Can be an Array (multiple records) or Object (single record)
- Must be parsed using methods like `.json()`, `.text()`, `.blob()`

```
const response = await fetch(url);
console.log(response.headers.get('content-type')); // Headers
const data = await response.json(); // Body
```

5. Why do we need to use .json() method on the fetch response?

Answer: The `.json()` method is needed because:

1. **Fetch returns raw response:** The response body is a readable stream, not parsed data
2. **Conversion to JavaScript:** `.json()` parses the JSON string into a JavaScript object
3. **Returns Promise:** The parsing is asynchronous, so it returns a Promise
4. **Human-readable format:** Without parsing, data isn't usable in JavaScript

```
const response = await fetch(url);
// response.body is a ReadableStream

const data = await response.json();
// Now data is a JavaScript object/array
```

6. What's the difference between async function and regular function when using fetch?

Answer:

```
// Regular function - Returns Promise
function getData() {
  return fetch(url).then(res => res.json());
}
// Usage: getData().then(data => console.log(data));

// Async function - Returns Promise but cleaner syntax
async function getData() {
  const res = await fetch(url);
  const data = await res.json();
  return data; // Still returns Promise
}
// Usage: const data = await getData();
```

Key Points:

- Both return Promises
- Async functions always wrap return values in Promises
- Async/await provides cleaner, more readable code
- Better error handling with try-catch

7. How do you handle errors in fetch API?

Answer:

```
async function fetchData(url) {
  try {
    const response = await fetch(url);

    // Fetch doesn't reject on HTTP errors (404, 500, etc.)
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    // Catches network errors and our thrown errors
    console.error('Error:', error.message);
    throw error;
  }
}
```

Important: Fetch only rejects on network failures, not HTTP errors. You must check `response.ok` or `response.status`.

8. What are HTTP status codes and what do they mean?

Answer:

1xx - Informational: Request received, continuing process

- 100 - Continue

2xx - Success: Request successfully received and accepted

- 200 - OK (Success)
- 201 - Created
- 204 - No Content

3xx - Redirection: Further action needed

- 301 - Moved Permanently
- 304 - Not Modified (cached)

4xx - Client Error: Request has an error

- 400 - Bad Request
- 401 - Unauthorized (not authenticated)
- 403 - Forbidden (authenticated but not allowed)
- 404 - Not Found
- 422 - Unprocessable Entity

5xx - Server Error: Server failed to fulfill request

- 500 - Internal Server Error
- 502 - Bad Gateway
- 503 - Service Unavailable

9. Explain the difference between PUT and PATCH methods.

Answer:

PUT:

- Replaces the entire resource
- Requires sending all fields
- Idempotent (same result if called multiple times)

```
// PUT - Must send all fields
await fetch(`/users/123`, {
  method: 'PUT',
  body: JSON.stringify({
    name: 'John',
    email: 'john@example.com',
    age: 30,
    city: 'New York'
  })
});
```

PATCH:

- Updates only specific fields
- Only send fields you want to change
- Also idempotent

```
// PATCH - Only send fields to update
await fetch(`/users/123`, {
  method: 'PATCH',
  body: JSON.stringify({
    email: 'newemail@example.com'
  })
});
```

10. What is CORS and how does it affect fetch requests?

Answer: CORS (Cross-Origin Resource Sharing) is a security mechanism that restricts web pages from making requests to a different domain than the one serving the page.

Same-Origin: `https://example.com/page` can fetch `https://example.com/api` **Cross-Origin:** `https://example.com` cannot fetch `https://api.other.com` (unless CORS is enabled)

CORS Headers (set by server):

```
Access-Control-Allow-Origin: * // Allow all domains
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: Content-Type
```

Handling CORS errors:

- Cannot be fixed on client side
- Server must set appropriate CORS headers
- Use proxy server in development
- Use `mode: 'cors'` in fetch options (default)

11. How do you send POST request with JSON data?

Answer:

```

async function createUser(userData) {
  const response = await fetch('https://api.example.com/users', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      // Add other headers if needed
      // 'Authorization': 'Bearer token123'
    },
    body: JSON.stringify(userData) // Convert object to JSON string
  });

  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }

  const data = await response.json();
  return data;
}

// Usage
const newUser = {
  name: 'John Doe',
  email: 'john@example.com',
  age: 30
};

const result = await createUser(newUser);
console.log(result);

```

Key Points:

- Set `Content-Type: application/json` header
- Use `JSON.stringify()` to convert object to JSON string
- Always check `response.ok` for errors

12. What is the difference between `response.ok` and `response.status`?

Answer:

`response.ok`:

- Boolean value
- `true` if status is 200-299 (success range)
- `false` for all other status codes

`response.status`:

- Numeric HTTP status code
- Examples: 200, 404, 500, 401, etc.

```
const response = await fetch(url);

console.log(response.ok);      // true or false
console.log(response.status);  // 200, 404, 500, etc.

// Usage
if (!response.ok) {
  if (response.status === 404) {
    console.log('Not found');
  } else if (response.status === 500) {
    console.log('Server error');
  }
}
```

13. How do you abort a fetch request?

Answer:

```
// Create AbortController
const controller = new AbortController();
const signal = controller.signal;

// Start fetch with signal
const fetchPromise = fetch(url, { signal });

// Abort after 5 seconds
setTimeout(() => {
  controller.abort();
  console.log('Request aborted');
}, 5000);

// Handle abort
try {
  const response = await fetchPromise;
  const data = await response.json();
  console.log(data);
} catch (error) {
  if (error.name === 'AbortError') {
    console.log('Fetch was aborted');
  } else {
    console.error('Fetch error:', error);
  }
}
```

Use Cases:

- User navigates away from page
- Implementing request timeout
- Cancelling outdated requests
- User cancels action (e.g., stops search)

14. What are the different ways to parse a fetch response?

Answer:

```

const response = await fetch(url);

// 1. JSON data
const jsonData = await response.json();

// 2. Plain text
const textData = await response.text();

// 3. Blob (files, images)
const blobData = await response.blob();

// 4. FormData
const formData = await response.formData();

// 5. ArrayBuffer (binary data)
const buffer = await response.arrayBuffer();

```

Examples:

```

// Image download
const response = await fetch('image.jpg');
const blob = await response.blob();
const imageUrl = URL.createObjectURL(blob);
document.getElementById('img').src = imageUrl;

// HTML content
const response = await fetch('page.html');
const html = await response.text();
document.body.innerHTML = html;

```

15. How do you implement caching with fetch API?

Answer:

```

// Cache modes
const response = await fetch(url, {
  cache: 'default' // Use browser's cache strategy
});

// Cache options:
// 'default' - Use HTTP cache rules
// 'no-store' - Never cache
// 'reload' - Always fetch from server
// 'no-cache' - Validate cache before using
// 'force-cache' - Use cache, even if stale
// 'only-if-cached' - Only use cache, fail if not cached

```

Manual caching implementation:

```

const cache = {};

async function fetchWithCache(url) {
  // Check if data is in cache
  if (cache[url]) {
    console.log('Using cached data');
    return cache[url];
  }

  // Fetch and cache
  console.log('Fetching from server');
  const response = await fetch(url);
  const data = await response.json();

  cache[url] = data;
  return data;
}

// With expiration time
const cacheWithExpiry = {};

async function fetchWithExpiringCache(url, expiryMs = 60000) {
  const now = Date.now();
  const cached = cacheWithExpiry[url];

  // Check if cache exists and not expired
  if (cached && now - cached.timestamp < expiryMs) {
    return cached.data;
  }

  // Fetch new data
  const response = await fetch(url);
  const data = await response.json();

  cacheWithExpiry[url] = {
    data: data,
    timestamp: now
  };

  return data;
}

```

16. What is the purpose of headers in fetch and how do you use them?

Answer:

Headers contain metadata about the request or response.

Common Request Headers:

```

await fetch(url, {
  headers: {
    'Content-Type': 'application/json', // Data format being sent
    'Authorization': 'Bearer token123', // Authentication
    'Accept': 'application/json',      // Expected response format
    'User-Agent': 'MyApp/1.0',         // Client identification
    'Accept-Language': 'en-US',        // Preferred language
    'Cache-Control': 'no-cache'        // Caching behavior
  }
});

```

Reading Response Headers:

```

const response = await fetch(url);

// Get specific header
const contentType = response.headers.get('content-type');
const cacheControl = response.headers.get('cache-control');

// Check if header exists
const hasAuth = response.headers.has('authorization');

// Iterate all headers
response.headers.forEach((value, key) => {
  console.log(`${key}: ${value}`);
});

```

17. Explain Promise chaining vs async/await with fetch.

Answer:

Promise Chaining (Traditional):

```

function getUserData(userId) {
  return fetch(`/users/${userId}`)
    .then(response => {
      if (!response.ok) {
        throw new Error('User not found');
      }
      return response.json();
    })
    .then(user => {
      console.log('User:', user);
      return fetch(`/users/${userId}/posts`);
    })
    .then(response => response.json())
    .then(posts => {
      console.log('Posts:', posts);
      return posts;
    })
    .catch(error => {
      console.error('Error:', error);
    });
}

```

Async/Await (Modern):

```

async function getUserData(userId) {
  try {
    // Fetch user
    const userResponse = await fetch(`/users/${userId}`);
    if (!userResponse.ok) {
      throw new Error('User not found');
    }
    const user = await userResponse.json();
    console.log('User:', user);

    // Fetch user's posts
    const postsResponse = await fetch(`/users/${userId}/posts`);
    const posts = await postsResponse.json();
    console.log('Posts:', posts);

    return posts;
  } catch (error) {
    console.error('Error:', error);
  }
}

```

Advantages of async/await:

- More readable and maintainable
- Easier error handling with try-catch
- Looks like synchronous code
- Better debugging experience

18. How do you make multiple fetch requests in parallel?

Answer:

Sequential (one after another):

```

async function fetchSequential() {
  const user = await fetch('/user/1').then(r => r.json());
  const posts = await fetch('/posts').then(r => r.json());
  const comments = await fetch('/comments').then(r => r.json());

  return { user, posts, comments };
}
// Takes: time(user) + time(posts) + time(comments)

```

Parallel (all at once):

```

async function fetchParallel() {
  const [user, posts, comments] = await Promise.all([
    fetch('/user/1').then(r => r.json()),
    fetch('/posts').then(r => r.json()),
    fetch('/comments').then(r => r.json())
  ]);

  return { user, posts, comments };
}
// Takes: max(time(user), time(posts), time(comments))

```

With error handling:

```

async function fetchParallelSafe() {
  try {
    const [userRes, postsRes, commentsRes] = await Promise.all([
      fetch('/user/1'),
      fetch('/posts'),
      fetch('/comments')
    ]);

    // Check all responses
    if (!userRes.ok || !postsRes.ok || !commentsRes.ok) {
      throw new Error('One or more requests failed');
    }

    // Parse all
    const [user, posts, comments] = await Promise.all([
      userRes.json(),
      postsRes.json(),
      commentsRes.json()
    ]);

    return { user, posts, comments };
  } catch (error) {
    console.error('Error:', error);
  }
}

```

Promise.allSettled (doesn't fail if one fails):

```

async function fetchAllSettled() {
  const results = await Promise.allSettled([
    fetch('/user/1').then(r => r.json()),
    fetch('/posts').then(r => r.json()),
    fetch('/comments').then(r => r.json())
  ]);

  const [userResult, postsResult, commentsResult] = results;

  const user = userResult.status === 'fulfilled' ? userResult.value : null;
  const posts = postsResult.status === 'fulfilled' ? postsResult.value : null;
  const comments = commentsResult.status === 'fulfilled' ? commentsResult.value : null;

  return { user, posts, comments };
}

```

Practical Coding Questions

Question 1: Create a function to fetch user data and display it

Task: Fetch a GitHub user's data and display their name, avatar, and follower count.

```
// HTML
<button id="fetchBtn">Get User Data</button>
<div id="userContainer"></div>

// JavaScript
const fetchBtn = document.getElementById('fetchBtn');
const container = document.getElementById('userContainer');

fetchBtn.addEventListener('click', async () => {
  try {
    const username = 'octocat'; // Change to any username
    const userData = await fetchGitHubUser(username);
    displayUser(userData);
  } catch (error) {
    container.innerHTML = `<p style="color: red;">Error: ${error.message}</p>`;
  }
});

async function fetchGitHubUser(username) {
  const url = `https://api.github.com/users/${username}`;
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`User not found (${response.status})`);
  }

  return await response.json();
}

function displayUser(user) {
  container.innerHTML = `
    <div class="user-card">
      
      <h2>${user.name} || user.login</h2>
      <p><strong>Username:</strong> ${user.login}</p>
      <p><strong>Followers:</strong> ${user.followers}</p>
      <p><strong>Following:</strong> ${user.following}</p>
      <p><strong>Public Repos:</strong> ${user.public_repos}</p>
      <a href="${user.html_url}" target="_blank">View GitHub Profile</a>
    </div>
  `;
}
```

Question 2: Implement search functionality with debouncing

Task: Create a search input that searches GitHub users as you type, with debouncing to avoid too many API calls.

```
// HTML
<input type="text" id="searchInput" placeholder="Search GitHub users...">
<div id="searchResults"></div>

// JavaScript
const searchInput = document.getElementById('searchInput');
const searchResults = document.getElementById('searchResults');

let debounceTimer;

searchInput.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);

  const query = e.target.value.trim();

  if (query.length < 2) {
    searchResults.innerHTML = '';
    return;
  }

  // Debounce - wait 500ms after user stops typing
  debounceTimer = setTimeout(() => {
    searchUsers(query);
  }, 500);
});

async function searchUsers(query) {
  searchResults.innerHTML = '<p>Searching...</p>';

  try {
    const url = `https://api.github.com/search/users?q=${query}&per_page=5`;
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error('Search failed');
    }

    const data = await response.json();
    displaySearchResults(data.items);
  } catch (error) {
    searchResults.innerHTML = `<p style="color: red;">Error: ${error.message}</p>`;
  }
}

function displaySearchResults(users) {
  if (users.length === 0) {
    searchResults.innerHTML = '<p>No users found</p>';
    return;
  }

  const html = users.map(user => `
    <div class="search-result">
      
      <div>
        <strong>${user.login}</strong>
        <a href="${user.html_url}" target="_blank">View Profile</a>
      </div>
    </div>
  `).join('');

  searchResults.innerHTML = html;
}
```

Question 3: Create a todo list with CRUD operations

Task: Implement a todo list that uses JSONPlaceholder API for CRUD operations.

```

const API_URL = 'https://jsonplaceholder.typicode.com/todos';
const todoList = document.getElementById('todoList');
const addBtn = document.getElementById('addBtn');
const todoInput = document.getElementById('todoInput');

// Load todos on page load
loadTodos();

async function loadTodos() {
  try {
    const response = await fetch(`${API_URL}?_limit=5`);
    const todos = await response.json();
    displayTodos(todos);
  } catch (error) {
    console.error('Error loading todos:', error);
  }
}

function displayTodos(todos) {
  const html = todos.map(todo => `
    <div class="todo-item" data-id="${todo.id}">
      <input type="checkbox" ${todo.completed ? 'checked' : ''}
        onchange="toggleTodo(${todo.id}, this.checked)">
      <span class="${todo.completed ? 'completed' : ''}">${todo.title}</span>
      <button onclick="deleteTodo(${todo.id})">Delete</button>
    </div>
  `).join('');

  todoList.innerHTML = html;
}

// CREATE
addBtn.addEventListener('click', async () => {
  const title = todoInput.value.trim();

  if (!title) {
    alert('Please enter a todo');
    return;
  }

  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title: title,
        completed: false,
        userId: 1
      })
    });

    const newTodo = await response.json();
    console.log('Created:', newTodo);
    todoInput.value = '';
    loadTodos(); // Refresh list
  } catch (error) {
    console.error('Error creating todo:', error);
  }
});

// UPDATE
async function toggleTodo(id, completed) {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'PATCH',

```

```
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ completed })
  });

  const updated = await response.json();
  console.log('Updated:', updated);
} catch (error) {
  console.error('Error updating todo:', error);
}
}

// DELETE
async function deleteTodo(id) {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'DELETE'
    });

    if (response.ok) {
      console.log('Deleted todo:', id);
      loadTodos(); // Refresh list
    }
  } catch (error) {
    console.error('Error deleting todo:', error);
  }
}
```

Question 4: Implement infinite scroll pagination

Task: Load more GitHub users as you scroll down the page.

```

const container = document.getElementById('usersContainer');
let currentPage = 1;
let isLoading = false;

// Initial load
loadUsers();

// Scroll event listener
window.addEventListener('scroll', () => {
  const { scrollTop, scrollHeight, clientHeight } = document.documentElement;

  // Check if user scrolled near bottom (within 100px)
  if (scrollTop + clientHeight >= scrollHeight - 100 && !isLoading) {
    loadUsers();
  }
});

async function loadUsers() {
  if (isLoading) return;

  isLoading = true;
  showLoadingIndicator();

  try {
    const perPage = 20;
    const since = (currentPage - 1) * perPage;
    const url = `https://api.github.com/users?since=${since}&per_page=${perPage}`;

    const response = await fetch(url);
    const users = await response.json();

    displayUsers(users);
    currentPage++;
  } catch (error) {
    console.error('Error loading users:', error);
    container.innerHTML += '<p style="color: red;">Error loading users</p>';
  } finally {
    isLoading = false;
    hideLoadingIndicator();
  }
}

function displayUsers(users) {
  const html = users.map(user => `
    <div class="user-card">
      
      <h3>${user.login}</h3>
      <a href="${user.html_url}" target="_blank">View Profile</a>
    </div>
  `).join('');

  container.innerHTML += html;
}

function showLoadingIndicator() {
  const loader = document.createElement('div');
  loader.id = 'loader';
  loader.innerHTML = '<p>Loading more users...</p>';
  document.body.appendChild(loader);
}

function hideLoadingIndicator() {
  const loader = document.getElementById('loader');
  if (loader) {
    loader.remove();
  }
}

```

```

}
}

```

Question 5: Implement request cancellation

Task: Cancel previous search request when user types new query.

```

let currentController = null;

async function searchWithCancellation(query) {
  // Cancel previous request if exists
  if (currentController) {
    currentController.abort();
    console.log('Previous request cancelled');
  }

  // Create new controller for this request
  currentController = new AbortController();
  const signal = currentController.signal;

  try {
    const url = `https://api.github.com/search/users?q=${query}`;
    const response = await fetch(url, { signal });

    if (!response.ok) {
      throw new Error('Search failed');
    }

    const data = await response.json();
    displayResults(data.items);

    // Clear controller after successful request
    currentController = null;
  } catch (error) {
    if (error.name === 'AbortError') {
      console.log('Request was cancelled');
    } else {
      console.error('Search error:', error);
    }
  }
}

// Usage with input
const searchInput = document.getElementById('search');
let debounceTimer;

searchInput.addEventListener('input', (e) => {
  clearTimeout(debounceTimer);

  const query = e.target.value.trim();

  if (query.length < 2) return;

  debounceTimer = setTimeout(() => {
    searchWithCancellation(query);
  }, 300);
});

```

Question 6: Implement retry logic with exponential backoff

Task: Retry failed requests with increasing delays.

```

async function fetchWithRetry(url, maxRetries = 3) {
  let lastError;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      console.log(`Attempt ${attempt + 1}/${maxRetries}`);

      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
      }

      return await response.json();
    } catch (error) {
      lastError = error;
      console.log(`Attempt ${attempt + 1} failed:`, error.message);

      // Don't wait after last attempt
      if (attempt < maxRetries - 1) {
        // Exponential backoff: 1s, 2s, 4s
        const delay = Math.pow(2, attempt) * 1000;
        console.log(`Waiting ${delay}ms before retry...`);
        await new Promise(resolve => setTimeout(resolve, delay));
      }
    }
  }

  throw new Error(`Failed after ${maxRetries} attempts: ${lastError.message}`);
}

// Usage
async function loadData() {
  try {
    const data = await fetchWithRetry('https://api.example.com/data', 3);
    console.log('Success:', data);
  } catch (error) {
    console.error('All retries failed:', error);
  }
}

```

Question 7: Upload file with progress tracking

Task: Upload a file and show upload progress.

```

async function uploadFile(file) {
  const formData = new FormData();
  formData.append('file', file);

  try {
    const response = await fetch('https://api.example.com/upload', {
      method: 'POST',
      body: formData,
      // Note: Don't set Content-Type header, browser will set it automatically with bo
    });

    if (!response.ok) {
      throw new Error('Upload failed');
    }

    const result = await response.json();
    return result;
  } catch (error) {
    console.error('Upload error:', error);
    throw error;
  }
}

// With XMLHttpRequest for progress tracking
function uploadFileWithProgress(file, onProgress) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    const formData = new FormData();
    formData.append('file', file);

    // Progress tracking
    xhr.upload.addEventListener('progress', (e) => {
      if (e.lengthComputable) {
        const percentComplete = (e.loaded / e.total) * 100;
        onProgress(percentComplete);
      }
    });

    xhr.addEventListener('load', () => {
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(JSON.parse(xhr.responseText));
      } else {
        reject(new Error(`Upload failed: ${xhr.status}`));
      }
    });

    xhr.addEventListener('error', () => {
      reject(new Error('Upload failed'));
    });

    xhr.open('POST', 'https://api.example.com/upload');
    xhr.send(formData);
  });
}

// Usage
const fileInput = document.getElementById('fileInput');
const progressBar = document.getElementById('progress');

fileInput.addEventListener('change', async (e) => {
  const file = e.target.files[0];

  if (!file) return;

  try {
    const result = await uploadFileWithProgress(file, (percent) => {

```

```
progressBar.style.width = `${percent}%`;
progressBar.textContent = `${Math.round(percent)}%`;
});

console.log('Upload successful:', result);
} catch (error) {
  console.error('Upload failed:', error);
}
});
```

Question 8: Implement data caching with localStorage

Task: Cache API responses in localStorage to reduce API calls.

```

const CACHE_PREFIX = 'api_cache_';
const CACHE_DURATION = 5 * 60 * 1000; // 5 minutes

async function fetchWithCache(url, forceRefresh = false) {
  const cacheKey = CACHE_PREFIX + url;

  // Check cache first
  if (!forceRefresh) {
    const cachedData = getFromCache(cacheKey);
    if (cachedData) {
      console.log('Using cached data');
      return cachedData;
    }
  }

  // Fetch from API
  console.log('Fetching from API');
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }

  const data = await response.json();

  // Save to cache
  saveToCache(cacheKey, data);

  return data;
}

function saveToCache(key, data) {
  const cacheObject = {
    data: data,
    timestamp: Date.now()
  };

  try {
    localStorage.setItem(key, JSON.stringify(cacheObject));
  } catch (error) {
    console.error('Failed to save to cache:', error);
  }
}

function getFromCache(key) {
  try {
    const cached = localStorage.getItem(key);

    if (!cached) {
      return null;
    }

    const cacheObject = JSON.parse(cached);
    const now = Date.now();

    // Check if cache is still valid
    if (now - cacheObject.timestamp < CACHE_DURATION) {
      return cacheObject.data;
    } else {
      // Cache expired, remove it
      localStorage.removeItem(key);
      return null;
    }
  } catch (error) {
    console.error('Failed to get from cache:', error);
    return null;
  }
}

```

```

}

function clearCache() {
  const keys = Object.keys(localStorage);
  keys.forEach(key => {
    if (key.startsWith(CACHE_PREFIX)) {
      localStorage.removeItem(key);
    }
  });
  console.log('Cache cleared');
}

// Usage
async function loadUserData(userId) {
  try {
    const url = `https://api.github.com/users/${userId}`;
    const data = await fetchWithCache(url);
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
}

// Load with force refresh
async function refreshUserData(userId) {
  const url = `https://api.github.com/users/${userId}`;
  const data = await fetchWithCache(url, true); // Force refresh
  console.log(data);
}

```

Question 9: Implement rate limiting

Task: Limit the number of API calls per minute.

```

class RateLimiter {
  constructor(maxRequests, timeWindow) {
    this.maxRequests = maxRequests; // Max requests allowed
    this.timeWindow = timeWindow; // Time window in milliseconds
    this.requests = []; // Timestamps of requests
  }

  async throttle() {
    const now = Date.now();

    // Remove old requests outside time window
    this.requests = this.requests.filter(
      timestamp => now - timestamp < this.timeWindow
    );

    // Check if limit reached
    if (this.requests.length >= this.maxRequests) {
      const oldestRequest = this.requests[0];
      const waitTime = this.timeWindow - (now - oldestRequest);

      console.log(`Rate limit reached. Waiting ${waitTime}ms...`);
      await new Promise(resolve => setTimeout(resolve, waitTime));

      // Retry
      return this.throttle();
    }

    // Add current request
    this.requests.push(now);
  }
}

// Create limiter: 10 requests per minute
const limiter = new RateLimiter(10, 60000);

async function fetchWithRateLimit(url) {
  await limiter.throttle();

  const response = await fetch(url);
  return await response.json();
}

// Usage
async function loadMultipleUsers(userIds) {
  for (const id of userIds) {
    try {
      const data = await fetchWithRateLimit(`https://api.github.com/users/${id}`);
      console.log(data);
    } catch (error) {
      console.error(`Error fetching user ${id}:`, error);
    }
  }
}

// Load 20 users (will automatically rate limit)
const userIds = Array.from({ length: 20 }, (_, i) => i + 1);
loadMultipleUsers(userIds);

```

Question 10: Build a weather app with geolocation

Task: Get user's location and fetch weather data.

```

const API_KEY = 'your_api_key_here';

async function getWeatherByLocation() {
  try {
    // Get user's coordinates
    const position = await getCurrentPosition();
    const { latitude, longitude } = position.coords;

    console.log(`Location: ${latitude}, ${longitude}`);

    // Fetch weather data
    const weather = await fetchWeather(latitude, longitude);
    displayWeather(weather);
  } catch (error) {
    console.error('Error:', error);
    document.getElementById('weather').innerHTML =
      `<p style="color: red;">Error: ${error.message}</p>`;
  }
}

function getCurrentPosition() {
  return new Promise((resolve, reject) => {
    if (!navigator.geolocation) {
      reject(new Error('Geolocation not supported'));
      return;
    }

    navigator.geolocation.getCurrentPosition(resolve, reject);
  });
}

async function fetchWeather(lat, lon) {
  const url = `https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=${API_KEY}&units=metric`;

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error('Failed to fetch weather data');
  }

  return await response.json();
}

function displayWeather(data) {
  const weatherDiv = document.getElementById('weather');

  weatherDiv.innerHTML = `
    <h2>Weather in ${data.name}</h2>
    <div class="weather-info">
      
      <p class="temp">${Math.round(data.main.temp)}°C</p>
      <p class="description">${data.weather[0].description}</p>
      <p>Humidity: ${data.main.humidity}%</p>
      <p>Wind Speed: ${data.wind.speed} m/s</p>
    </div>
  `;
}

// Load on page load
document.addEventListener('DOMContentLoaded', () => {
  const loadBtn = document.getElementById('loadWeatherBtn');
  loadBtn.addEventListener('click', getWeatherByLocation);
});

```

Best Practices Summary

1. Always Handle Errors

```
try {
  const response = await fetch(url);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
} catch (error) {
  console.error('Error:', error);
}
```

2. Check Response Status

```
if (!response.ok) {
  throw new Error(`HTTP error! status: ${response.status}`);
}
```

3. Use Async/Await

```
// Good
async function getData() {
  const res = await fetch(url);
  return await res.json();
}

// Avoid
function getData() {
  return fetch(url).then(res => res.json());
}
```

4. Set Appropriate Headers

```
await fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(data)
});
```

5. Implement Loading States

```
container.innerHTML = 'Loading...';
const data = await fetch(url);
// Display data
```

6. Use Timeouts for Long Requests

```
const controller = new AbortController();
setTimeout(() => controller.abort(), 5000);

await fetch(url, { signal: controller.signal });
```

7. Cache When Appropriate

```
// Use cache for static or slow-changing data
const response = await fetch(url, { cache: 'force-cache' });
```

8. Validate User Input

```
const query = input.value.trim();
if (!query) {
  alert('Please enter a search term');
  return;
}
```

9. Clean Up Event Listeners

```
const controller = new AbortController();

window.addEventListener('scroll', handleScroll, {
  signal: controller.signal
});

// Later: controller.abort();
```

10. Use Environment Variables for API Keys

```
// Never hardcode API keys
const API_KEY = process.env.API_KEY;
// or
const API_KEY = import.meta.env.VITE_API_KEY;
```

Common Pitfalls to Avoid

1. Not checking response.ok

```
// Wrong
const data = await fetch(url).then(r => r.json());

// Correct
const response = await fetch(url);
if (!response.ok) throw new Error('Failed');
const data = await response.json();
```

2. Forgetting to parse response

```
// Wrong
const data = await fetch(url);

// Correct
const response = await fetch(url);
const data = await response.json();
```

3. Not handling network errors

```
// Wrong
const data = await fetch(url).then(r => r.json());

// Correct
try {
  const data = await fetch(url).then(r => r.json());
} catch (error) {
  console.error('Network error:', error);
}
```

4. Making too many sequential requests

```
// Slow
const user = await fetch(url1);
const posts = await fetch(url2);

// Fast
const [user, posts] = await Promise.all([
  fetch(url1),
  fetch(url2)
]);
```

5. Not using AbortController for cancellation

```
// No way to cancel  
fetch(url);  
  
// Can be cancelled  
const controller = new AbortController();  
fetch(url, { signal: controller.signal });  
controller.abort();
```

Additional Resources

- [MDN Fetch API Documentation](#)
- [JavaScript.info - Fetch](#)
- [HTTP Status Codes](#)
- [JSONPlaceholder - Fake API for testing](#)
- [GitHub API Documentation](#)

End of Notes